

(1) 3.3.1

The prompt:

Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. We are a team of 5 and we are more familiar with python and java.

The AI Response:

The updated class diagram provides a highly specific architectural blueprint that strongly favors a **Python-based ecosystem**. The inclusion of "Blueprints" in the controller layers and the reliance on specific AI services (DeepSeek, Ollama) point toward a modular Python web framework.

Recommended Technology Stack

Layer	Technology	Rationale
Frontend	React.js	Best for building the dynamic dashboard and handling the industry/course selection state (FR2, FR3) while meeting the 1-second filter update requirement (NFR1.3).
Backend	Flask	The diagram explicitly organizes controllers using the "Blueprint" pattern (auth_controller, resume_controller, etc.), which is a core architectural concept of Flask.
Database	PostgreSQL	Necessary to support the relational complexity between users, resume analyses, and favorite courses, as well as the unique constraints (e.g., email being unique).
AI / NLP	LangChain	Integrates perfectly with the SkillExtractionService and AIScorer, allowing you to swap between DeepSeek API and local Ollama instances as designed.

Data Handling	Pandas	The most efficient way to implement the DataService logic for downloading and processing large Job Vacancy CSVs and SkillsFuture Excel files (FR1.1, FR1.2).
---------------	--------	--

Architectural Alignment with Your Design

1. Modular Controller Structure

The diagram defines clear controller boundaries (auth_controller, resume_controller, dashboard_controller, and career_controller). In **Flask**, these are implemented as **Blueprints**, which allow your team of five to work on different feature sets (e.g., one person on authentication, another on resume logic) without causing significant code conflicts.

2. The AI & Resume Logic

The design includes a sophisticated grading and extraction pipeline:

- **SkillExtractionService**: Using Python allows you to use libraries like PyMuPDF or pdfplumber for the extract_text_from_pdf method mentioned in your ResumeAnalysisService.
- **AI Scorer & RuleBasedScorer**: Python's flexibility makes it easy to implement the "fallback" logic where the AI Scorer can drop back to rule-based scoring if the AI service is offline or slow.

3. Data Ingestion Performance

To meet the 5-second dashboard load requirement (**NFR1.1**), your **DataService** should not parse the CSV/Excel files on every request. Instead:

- Use a Python background task (like **Celery** or a simple cron script) to run the download_job_vacancies_csv method.
- Store the processed results in **PostgreSQL** so the get_dashboard_stats method only needs to perform a quick SQL query rather than a file read.

Critical Observation: Security & Data Conflict

There is a significant contradiction between your requirements and your diagram that your team should resolve before coding:

The Conflict:

- **NFR 5.1** explicitly states: "System must **not store** any personal user data."
- **The Class Diagram** includes a User entity with email, password_hash, and full_name, and a ResumeAnalysis entity that stores filename and feedback_text.

Recommendation: If the "no storage" requirement (NFR 5.1) is the priority, you must

remove the User, UserSkill, and ResumeAnalysis tables from your database design and rely on a session-based or "guest" experience. If the application requires accounts (as the diagram suggests), you must update NFR 5.1 to reflect a secure PII (Personally Identifiable Information) storage policy.

3.3.2 Answers

The AI's recommendations are mostly the same as in the previous exercise. In both cases, it suggests a Python-based backend, React for the frontend, PostgreSQL for the database, and Pandas for data processing. The main difference is the shift from FastAPI to Flask as the backend framework.

This change is likely due to differences in the updated class diagram rather than changes in the prompt. The new diagram introduces a clear controller structure (e.g., `auth_controller`, `resume_controller`) and follows a modular design that closely matches Flask's Blueprint pattern. It also includes more detailed AI components, such as `SkillExtractionService` and `AIScorer`, which led the AI to recommend a more structured AI integration (e.g., LangChain). These additions made Flask a more suitable choice for the architecture.

The AI appears to partially agree with the change in framework. It supports Flask because it aligns well with the modular design and team-based development. However, FastAPI may still be better in terms of performance and modern API features. Therefore, the change is reasonable from a design perspective, even if FastAPI remains technically stronger.

(2)3.3.3

The prompt:

generate a minimal skeleton template for project base on class fiagram
consisting of a folder hierarchy containing files that are empty except for TODO
comments.Do not to put any code inside the files

The AI Response:

This project skeleton is organized into a modular directory structure (Models, Services, and Controllers) based on the updated class diagram provided.

Project Skeleton Template

Folder Hierarchy

Plaintext

```
skillsfuture-app/  
├── controllers/  
│   ├── auth_controller.py  
│   ├── career_controller.py  
│   ├── dashboard_controller.py  
│   └── resume_controller.py  
├── models/  
│   ├── course.py  
│   ├── dashboard_stats.py  
│   ├── favorite_course.py  
│   ├── industry.py  
│   ├── resume_analysis.py  
│   ├── user.py  
│   └── user_skill.py  
├── services/  
│   ├── ai_scorer.py  
│   ├── authentication_service.py  
│   ├── data_service.py  
│   ├── resume_analysis_service.py  
│   ├── resume_editor_service.py  
│   ├── resume_grading_service.py  
│   └── rule_based_scorer.py
```

```
| └─ skill_extraction_service.py
└─ main.py
```

File Contents (TODOs)

The following files contain only implementation notes for the attributes and methods specified in the diagram.

Models Directory

- **user.py:** # TODO: Implement User model (id, email, password_hash, full_name, created_at, last_login, current_resume_path, resume_uploaded_at) and methods: set_password, check_password, update_last_login, add_favorite_course, remove_favorite_course, get_favorites, get_latest_analysis, etc.
- **user_skill.py:** # TODO: Implement UserSkill model (id, user_id, skill, category, source, extracted_at)
- **resume_analysis.py:** # TODO: Implement ResumeAnalysis model (id, user_id, filename, score, feedback, action_verbs_found, analyzed_at, various grade fields) and get_grade_breakdown()
- **industry.py:** # TODO: Implement Industry model (name, vacancies, growth_rate, icon) and get_growth_display()
- **course.py:** # TODO: Implement Course model (title, provider, duration, mode, rating, reviews, prices, subsidy_text, impact_level, skills, etc.) and get_subsidy_percentage()
- **favorite_course.py:** # TODO: Implement FavoriteCourse model (id, user_id, course_id, course_title, course_provider, course_language, added_at)
- **dashboard_stats.py:** # TODO: Implement DashboardStats model (total_vacancies, top_industries, total_courses)

Services Directory

- **authentication_service.py:** # TODO: Implement logic for register_user and authenticate_user
- **skill_extraction_service.py:** # TODO: Implement logic for extract_skills, extract_with_deepseek, extract_with_fallback, and is_ollama_running
- **resume_editor_service.py:** # TODO: Implement logic for file uploads, PDF data extraction, generating PDFs, and parsing resume sections/dates
- **resume_analysis_service.py:** # TODO: Implement logic for analyze_resume,

extract_text_from_pdf, and deepseek API integration

- **resume_grading_service.py:** # TODO: Implement grade() logic delegating to AI and Rule-based scorers
- **ai_scorer.py:** # TODO: Implement scoring logic for positions of responsibility, competencies, and extracurriculars using AI prompts
- **rule_based_scorer.py:** # TODO: Implement rule-based scoring for action verbs, specificity, avoided words, page count, and spell checks
- **data_service.py:** # TODO: Implement logic for downloading job/course CSVs, refreshing data, and filtering course/industry results

Controllers Directory

- **auth_controller.py:** # TODO: Define routes for login, register, and logout
 - **career_controller.py:** # TODO: Define routes for industry index and industry_detail
 - **dashboard_controller.py:** # TODO: Define routes for dashboard index, adding/removing favorites, and viewing analyses
 - **resume_controller.py:** # TODO: Define routes for upload, edit, save, download, results, and deletion
-

3.3.4 Answers:

The AI's output demonstrates a strong understanding of software architecture and largely aligns with the intent of the prompt. I agree to a great extent with its choice of organizing the project into *models*, *services*, and *controllers*, as this reflects a clean separation of concerns and is consistent with common design patterns such as MVC. The structure is logical, modular, and scalable, making it suitable for a real-world application derived from a class diagram.

However, the response does not fully adhere to the requirement of generating a *minimal skeleton*. While it correctly avoids adding any actual implementation code, the TODO comments are overly detailed. Instead of being simple placeholders, they include extensive descriptions of attributes, methods, and even system behaviour. This goes beyond what was requested and shifts the output from a minimal template to a semi-documented design specification.

Furthermore, the AI appears to have introduced additional assumptions, such as specific AI integrations (e.g., DeepSeek and Ollama) and data handling features, which may not have been explicitly present in the class diagram. This suggests that the output is not purely derived from the given input, but partially inferred, which reduces its precision.

On the other hand, the AI did not miss any major structural components, and the overall output remains reasonable and usable. It successfully captures the key entities and layers expected in such a system.

In conclusion, while the AI's response is technically sound and well-structured, it does not strictly satisfy the requirement for minimalism. It over-delivers by adding detailed TODO specifications and inferred features, making it slightly misaligned with the prompt despite being of high overall quality.